CAMPAIGN RESULTS

Mixed Training Campaign Results Report

Mixed Feedforward Campaign Closing Report

Styled PDF edition generated from the canonical Markdown analysis report for improved readability, section hierarchy, and table presentation.

Overview

This report summarizes the completed mixed feedforward training campaign prepared in:

- `doc/reports/campaign_plans/2026-03-12-15-32-28_mixed_training_campaign_plan_report.md`

The campaign was designed to test whether the next meaningful improvement over the earlier feedforward baseline family would come from:

- denser point sampling;
- larger effective batch sizes;
- larger model capacity;
- or interactions between those levers on top of the strongest long training schedule.

The campaign executed successfully end to end:

- completed runs: `9`
- failed runs: `0`
- queue status after completion: no pending, running, or failed campaign items

The execution artifact root is:

- `output/training_campaigns/2026-03-12-18-52-30_mixed_training_campaign_2026_03_12_15_32_28/`

Historical Reference Set

Before this mixed campaign, the feedforward project already had five important reference configurations:

- `trial`
- `baseline`
- `high_density`
- `high_epoch`
- `high_compute`

These runs remain useful as context because they define the baseline tradeoffs that motivated the mixed campaign.

Config	Status	Best Epoch	Approx. Wall Time	Val MAE [deg]	Val RMSE [deg]	Test MAE [deg]	Test RMSE [deg]
`trial`	historical reference	3	~5.0 min	0.003757	0.004555	0.003583	0.004295
`baseline`	historical reference	22	~11.2 min	0.003148	0.003655	0.003504	0.003969
`high_density`	historical reference	41	~12.6 min	0.003077	0.003642	0.003519	0.004046
`high_epoch`	historical reference	94	~24.2 min	0.003007	0.003451	0.003335	0.003767
`high_compute`	historical reference	21	~10.6 min	0.003198	0.003790	0.003319	0.003915

The earlier campaign already showed one clear result:

- longer optimization helped more than denser point sampling alone.

  That is why the new campaign kept the long schedule and explored density, batch, and capacity combinations around it.

Mixed Campaign Setup

The `9` new runs were organized in three phases:

- Phase 1

  Density plus long schedule.
- Phase 2

  Density plus long schedule plus larger batch.
- Phase 3

  Density plus long schedule plus larger batch plus larger model. All runs kept the same:
- dataset split logic;
- feature set (`input_size = 5`);
- target definition;
- optimizer family (`AdamW`);
- held-out validation and test workflow.

Phase 1 Results

Phase 1 asked a focused question

- if we keep the strongest long schedule, does denser sampling alone become enough to beat the historical `high_epoch` reference?

Config	Best Epoch	Wall Time	Val MAE [deg]	Val RMSE [deg]	Test MAE [deg]	Test RMSE [
`stride10_long`	107	23.8 min	0.003053	0.003588	0.003483	0.004050
`stride5_long`	43	17.2 min	0.003178	0.003660	0.003580	0.004008
`stride1_long`	66	37.7 min	0.003126	0.003514	0.003646	0.003990

Interpretation

Phase 1 did not overturn the earlier conclusion.

- `stride10_long` was the best Phase 1 candidate, but it still stayed behind `high_epoch` on held-out test metrics.
- `stride5_long` did not justify its stronger density setting.
- `stride1_long` was the most expensive Phase 1 run and also the weakest on `test_mae` .

  The practical message is still the same:
- denser sampling alone is not enough;
- especially full-density training is not automatically worth the cost.

Phase 2 Results

Phase 2 asked the next question

- if density is paired with a larger batch regime, do the denser variants become more competitive?

Config	Best Epoch	Wall Time	Val MAE [deg]	Val RMSE [deg]	Test MAE [deg]	Test RMSE [
`stride10_long_large_batch`	111	18.4 min	0.003066	0.003765	0.003433	0.004123
`stride5_long_large_batch`	81	16.8 min	0.003109	0.003639	0.003301	0.003791
`stride1_long_large_batch`	50	29.2 min	0.003104	0.003551	0.003358	0.003769

Interpretation

Phase 2 is the strongest part of the whole campaign.

- `stride5_long_large_batch` achieved the best `test_mae` not only within the mixed campaign, but across the full feedforward set currently available in the repository.
- `stride1_long_large_batch` produced a very strong `test_rmse`, nearly tied with the historical `high_epoch` reference.
- `stride10_long_large_batch` did not benefit enough from the larger batch to justify becoming the new default.

The important takeaway is:

- a larger batch helped when paired with the denser settings;
- the best practical gain came from the middle density regime (`stride = 5`), not from the full-density extreme.

Phase 3 Results

Phase 3 asked the final question

- after strengthening density and batch, does larger model capacity finally become worth it?

Config	Best Epoch	Wall Time	Val MAE [deg]	Val RMSE [deg]	Test MAE [deg]	Test RMSE [
`stride10_long_large_batch_big_model`	67	19.8 min	0.003040	0.003702	0.003413	0.004063
`stride5_long_large_batch_big_model`	104	33.8 min	0.003104	0.003666	0.003472	0.004004
`stride1_long_large_batch_big_model`	78	56.7 min	0.003090	0.003585	0.003308	0.003779

Interpretation

Phase 3 confirms that larger capacity is still not the main answer.

- the best large-model variant was `stride1_long_large_batch_big_model`, but it did not produce a decisive gain over the simpler Phase 2 leaders.
- `stride5_long_large_batch_big_model` was slower and worse than `stride5_long_large_batch`.
- `stride1_long_large_batch_big_model` remained very competitive on `test_mae`, but at a much larger runtime cost.

  The larger model therefore looks unnecessary as the default feedforward setting at this stage.

Cross-Campaign Ranking

The completed campaign clarifies the top of the feedforward ranking.

Best Test MAE

Config	Test MAE [deg]	Test RMSE [deg]	Runtime
`stride5_long_large_batch`	0.003301	0.003791	16.8 min
`stride1_long_large_batch_big_model`	0.003308	0.003779	56.7 min
`high_compute`	0.003319	0.003915	10.6 min
`high_epoch`	0.003335	0.003767	24.2 min
`stride1_long_large_batch`	0.003358	0.003769	29.2 min

Best Test RMSE

Config	Test RMSE [deg]	Test MAE [deg]	Runtime
`high_epoch`	0.003767	0.003335	24.2 min
`stride1_long_large_batch`	0.003769	0.003358	29.2 min
`stride1_long_large_batch_big_model`	0.003779	0.003308	56.7 min
`stride5_long_large_batch`	0.003791	0.003301	16.8 min
`high_compute`	0.003915	0.003319	10.6 min

Main Conclusions

The mixed campaign supports five practical conclusions.

1. The Long Schedule Remains Mandatory

Nothing in the new campaign argues for returning to the shorter baseline schedule. The long optimization regime remains the right backbone for serious feedforward training.

2. Full-Density Training Is Not The Best Default

`stride = 1` can be competitive, but it is expensive and not decisively superior. The full-density regime should remain a targeted benchmark, not the standard configuration.

3. The Middle-Density Regime Is The Best Practical Upgrade

`stride5_long_large_batch` is the most convincing practical winner:

- best global `test_mae`;
- much lower runtime than `high_epoch`;
- standard model capacity;
- no reliance on the heavier large-model architecture.

4. Larger Batch Helped More Than Larger Model Capacity

The strongest gains came from the Phase 2 batch increase, not from the Phase 3 model enlargement.

5. The Historical `high_epoch` Reference Still Matters

`high_epoch` remains slightly best on `test_rmse`. This means it is still a valuable secondary reference whenever the project wants the cleanest low-variance historical baseline.

Recommended Best Training Preset

The new recommended practical feedforward preset should be:

- `config/training/feedforward/presets/best_training.yaml`

  This preset should follow `stride5_long_large_batch`. Why this is the best current practical choice:
- it achieved the best held-out `test_mae` across the current feedforward configurations;
- it kept the standard `128-128-64` architecture;
- it avoided the extra cost and complexity of the larger model;
- it finished faster than `high_epoch` while still improving the main absolute-error metric.

  Important nuance:
- if the project later decides that `test_rmse` must dominate the ranking more strongly than `test_mae`, `high_epoch` remains the cleanest alternative reference.

Recommended Next Experiments

The next useful feedforward experiments are

1. repeat `best_training` with multiple random seeds;
2. compare `best_training` and `high_epoch` across multiple seeds to separate real gain from split-level variance;
3. perform regime-wise analysis by speed, torque, and temperature;
4. test whether a slightly lower learning rate further improves the `stride5_long_large_batch` convergence profile;
5. keep the large-model variants as optional offline benchmarks rather than default training presets.

Artifact References

Campaign-level references

- `output/training_campaigns/2026-03-12-18-52-30_mixed_training_campaign_2026_03_12_15_32_28/campaign_manifest.yaml`
- `output/training_campaigns/2026-03-12-18-52-30_mixed_training_campaign_2026_03_12_15_32_28/campaign_execution_report.md`

Best practical mixed-campaign run:

- `output/feedforward_network/te_feedforward_stride5_long_large_batch/`

Strong historical comparison run:

- `output/feedforward_network/te_feedforward_high_epoch/`

Each run folder includes:

- `best_checkpoint_path.txt`
- `training_test_metrics.yaml`
- `training_test_report.md`
- checkpoint files